

/explain

```
4
5 export function getModel {
6   const provider = PROVIDERS
7   if (!provider) {
8     throw new Error("Unknown provider")
9   }
10
11   const apiKey = process.env[provider.apiKeyEnvVar]
12   if (!apiKey) {
13     throw new Error(`${provider.name} API key is missing`)
14   }
15
16   const model = modality === "vision" ? provider.visionModel : provider.textModel
17   logInfo("Model", `Using ${provider.name} ${model}`)
18   return provider.build(apiKey, model)
19 }
```

`getModel` is a factory function that returns a chat model instance for either text or vision generation. It starts by looking up the configured provider from `PROVIDERS` using `env.activeProvider`. If that provider is not defined, it throws an error so the caller knows the environment is misconfigured.

Next, it reads the provider-specific API key from `process.env` using the provider's `apiKeyEnvVar`. If the key is missing, it throws another error, ensuring the app fails fast instead of trying to use an unauthenticated provider.

The function then chooses which model name to use based on the `modality` argument. If `modality` is `"vision"`, it picks `provider.visionModel`; otherwise it picks `provider.textModel`. That makes the same provider able to supply different models for text and image tasks.

Before returning the model, it logs the chosen provider and model with `logInfo`, which prints a timestamped message to the console. Finally, it calls `provider.build(apiKey, model)` and returns the resulting `BaseChatModel` instance.

The `Modality` type is a simple union of `"text"` or `"vision"`, so the caller must pass one of those two values.

/help

Help

I can assist with:

- Explaining the backend flow and how `modelFactory.ts` chooses an LLM provider
- Debugging route behavior in `chatRoute.ts` or `testCaseRoute.ts`
- Verifying moderation masking/blocking logic in `moderationService.ts`
- Writing or updating tests in `backend/src/**/*.test.ts`
- Clarifying environment config in `env.ts`

If you want, tell me:

- what behavior you want to inspect,
- which file or route,
- whether you need a code fix, test, or explanation.

/compact

Compacted conversation.

whats the change after /compact

Change after `/compact`

No code was changed by `/compact`.

It just produced a condensed session summary of the current state:

- your backend moderation flow and `/api/chat` integration
- the relevant files: `moderationService.ts`, `chatRoute.ts`, `chatRoute.test.ts`
- the issue with PII masking and the decision flow
- the fact that route tests were added/validated

So the "change" is only in the conversation context summary, not in the workspace files.

`/create-prompt`

`/create-prompt` create a readme.md file after analysing the current project

Reviewing existing workspace prompt conventions before creating the `.prompt.md` file.

Searched for files matching `**/*.prompt.md`, no matches

I've confirmed no existing prompt file exists. Creating a workspace prompt for README generation now.

Created `.prompt.md`

✓ Created `.prompt.md` in the repository root.

The prompt is designed to generate a `README.md` from the current project structure, backend/frontend setup, and environment configuration.

/context

/context

Context for `modelFactory.ts`

This file is the backend model factory that chooses and builds the chat model used for generation.

What it does

- Reads `env.activeProvider` to pick one of the configured providers.
- Uses the provider's required API key environment variable:
 - `GROQ_API_KEY`
 - `OPENAI_API_KEY`
 - `ANTHROPIC_API_KEY`
 - `MISTRAL_API_KEY`
- Chooses either the text or vision model name based on the `modality` argument.
- Logs the provider/model selection.
- Returns a LangChain `BaseChatModel` instance via `provider.build(...)`.

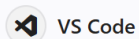
Supported providers

- `groq`
- `openai`



/search

@vscode /search getModel



Search the workspace for the identifier `getModel`.

Search